

Intelligence Artificielle : TD 1

Exercice 1

Règles :

- 4 pièces sont placées sur la table.
- Deux joueurs jouent à tour de rôle (MAX commence).
- À chaque tour, un joueur peut retirer 1 ou 2 pièces.
- Le joueur qui prend la dernière pièce gagne.

Questions :

1. Comment représenter un état du jeu ?
2. Quelles sont les actions possibles dans chaque état ?
3. Quel est l'état terminal ?
4. Quelle fonction d'utilité peut-on définir pour MIN et MAX ?
5. Construisez l'arbre de jeu Minimax.
6. MAX peut-il forcer une victoire ?

Exercice 2

Un utilisateur cherche un fichier nommé `report.pdf` sur son ordinateur.

On part du dossier racine `Home`. Chaque dossier peut contenir :

- des fichiers ;
- des sous-dossiers.

Pour retrouver le fichier, on explore l'arborescence des dossiers à partir de `Home`.

Structure donnée :

- `Home` contient les dossiers `Documents`, `Images`, `Téléchargements`
- `Documents` contient les dossiers `Cours` et `Administratif`
- `Images` contient les dossiers `Vacances` et `Travail`
- `Téléchargements` contient les dossiers `PDF` et `Archives`
- `Cours` contient les fichiers `notes.txt` et `slides.pdf`
- `Administratif` contient les fichiers `facture.pdf` et `cv.docx`
- `Vacances` contient les fichiers `photo1.jpg` et `photo2.jpg`
- `Travail` contient les fichiers `draft.docx` et `diagram.png`
- `PDF` contient les fichiers `paper.pdf` et `report.pdf`
- `Archives` contient les fichiers `old.zip` et `backup.tar`

Questions :

1. Modélisez ce problème sous forme d'arbre de recherche.
2. Quel est l'état initial ?
3. Quels sont les états successeurs de **Home** ?
4. Quel est l'état but ?
5. Construisez l'arbre correspondant.
6. Donnez l'ordre d'exploration des nœuds avec un parcours en largeur (BFS).
7. Donnez l'ordre d'exploration des nœuds avec un parcours en profondeur (DFS), de gauche à droite.
8. Avec quel algorithme trouve-t-on le fichier **report.pdf** en premier dans cet exemple ?

Exercice 3

Un médecin cherche à établir un diagnostic à partir de symptômes initiaux observés chez un patient. Cependant, il ne peut pas connaître immédiatement toutes les possibilités diagnostiques dès le départ.

En pratique :

- certaines hypothèses ne deviennent plausibles qu'après un premier examen ;
- certains tests ou examens orientent vers de nouveaux diagnostics ;
- le médecin découvre progressivement les options disponibles à mesure qu'il avance dans le raisonnement clinique.

Ainsi, le problème peut être représenté sous forme de graphe :

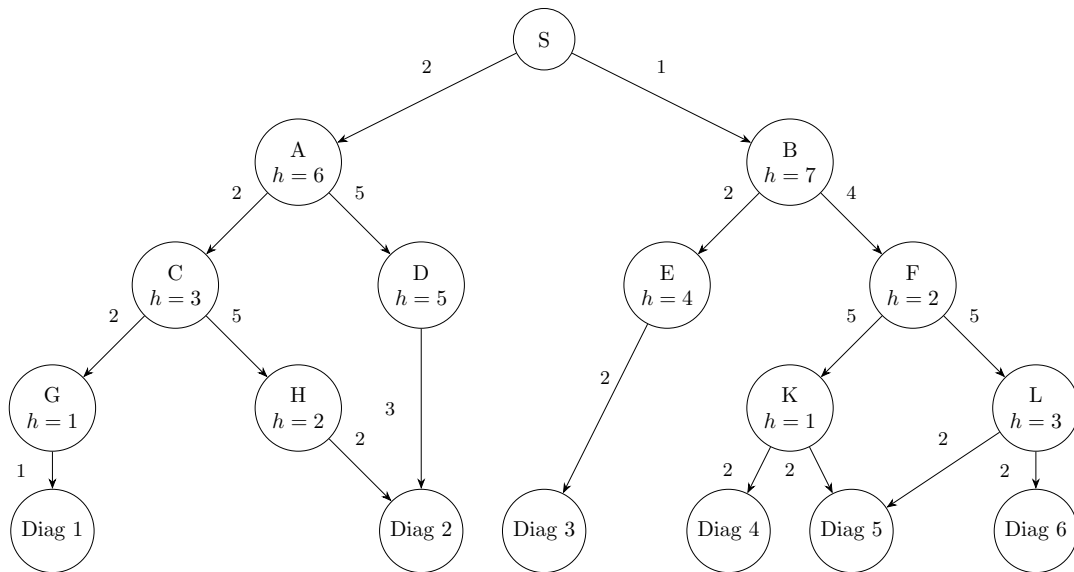
- chaque nœud représente un état de raisonnement médical ;
- chaque arête représente une transition vers une nouvelle hypothèse ou un nouvel examen ;
- chaque transition possède un **score de coût entier**, représentant par exemple :
 - le coût d'un examen,
 - le temps nécessaire,
 - le niveau de difficulté ou de risque,
 - ou plus généralement le “prix” de cette décision dans le processus de diagnostic.

L'objectif du médecin est d'atteindre un diagnostic final en minimisant le coût total du raisonnement.

Modélisation du problème :

- **État initial** : patient avec symptômes initiaux observés.
- **États intermédiaires** : hypothèses diagnostiques partielles ou résultats d'examens.
- **États terminaux** : diagnostics finaux possibles.
- **Actions** : prescrire un examen, interpréter un résultat, retenir une hypothèse.
- **Coût d'une action** : score entier associé à la transition.
- **Objectif** : atteindre un état terminal avec le coût total le plus faible.

On suppose qu'à chaque nœud est associée une valeur heuristique h , déterminée en fonction de la probabilité estimée. On obtient alors l'arbre suivant :

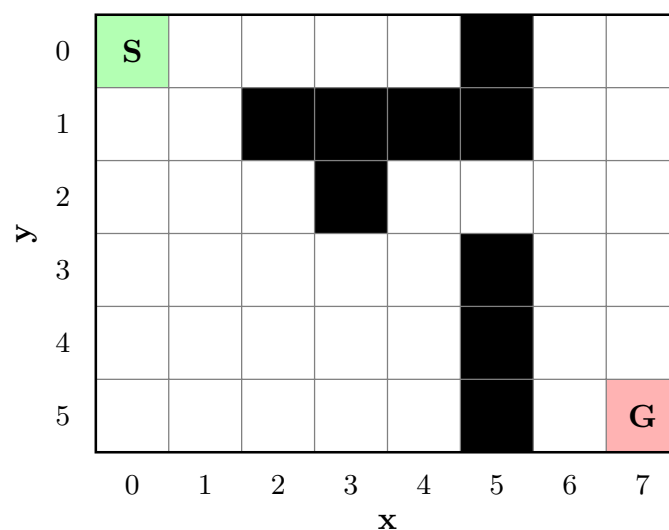


Questions :

1. Appliquez l'algorithme **Uniform Cost Search (UCS)** à partir de l'état initial.
2. Donnez l'ordre d'expansion des nœuds par UCS.
3. Quel diagnostic final UCS atteint-il en premier ?
4. Quel est le coût total de la solution trouvée par UCS ?
5. Appliquez ensuite l'algorithme **Best-First Search** sur le même graphe.
6. Donnez l'ordre d'expansion des nœuds par Best-First Search.
7. Quel diagnostic final Best-First Search atteint-il ?
8. Comparez les deux états terminaux obtenus :
 - sont-ils identiques ?
 - lequel correspond au coût total le plus faible ?
 - quel algorithme semble le plus rapide ? lequel garantit la meilleure solution ?

Exercice 4

Objectif : trouver le plus court chemin entre **S** et **G** dans la grille suivante à l'aide de l'algorithme A*.



Contraintes :

- Les cases noires sont des obstacles.
- On peut se déplacer uniquement en haut, bas, gauche ou droite.
- Les déplacements diagonaux sont interdits.
- L'heuristique imposée est la **distance de Manhattan** :

$$h(n) = |x_n - x_G| + |y_n - y_G|$$

Travail demandé :

1. Représenter les états et les actions possibles.
2. Calculer les valeurs $g(n)$, $h(n)$ et $f(n) = g(n) + h(n)$.
3. Exécuter A* jusqu'à atteindre l'objectif.
4. Donner le chemin final.
5. Donner le coût total du chemin.
6. Indiquer combien d'étapes ont été nécessaires pour trouver la solution.

Exercice 5

1. Combien de coups sont possibles au premier tour d'une partie de tic-tac-toe ? Combien le sont au deuxième tour ?
2. En supposant qu'aucune partie ne s'arrête avant que la grille ne soit complètement remplie, déduisez le nombre total de parties possibles.
3. On constate ainsi que, même pour un jeu simple, le nombre de configurations devient rapidement très grand. Déterminez la profondeur maximale d'un arbre de jeu contenant au plus 600 feuilles.
4. En utilisant la position de tic-tac-toe donnée ci-dessous, appliquez l'algorithme Minimax en considérant *O* comme joueur MAX et *X* comme joueur MIN.
5. MAX peut-il forcer une victoire, ou au moins un match nul ?

